

PROJECT SOUNDING LINE

The Software Verification and Parallel Test Infrastructure System

PART III

Repository, Codex, and Release Enforcement

Version 1.1 Amendment Edition

Amendment purpose

Make the Part I semantic-evidence and Part II queueing rules binding for repository tasks, policy changes, acceptance decisions, and closure records, without weakening any mandatory verification obligation.

Derived under owner authorization from the enacted Part I and Part II Version 1.1 amendments.

Effective August 10, 2026 | Document ID CS-SL-P3-001 v1.1

A. Document Control and Amendment Authority

Field	Value
Program	Project Sounding Line
Subsystem	The Software Verification and Parallel Test Infrastructure System
Document	Governing Architecture and Product Requirements Document - Part III, Version 1.1 Amendment Edition
Base document	Version 1.0, July 28, 2026
Version	1.1
Effective date	August 10, 2026
Document ID	CS-SL-P3-001 v1.1
Status	Governing amendment; additive and superseding only where clauses are explicitly named
Derivation	Produced under owner authorization from the enacted Part I and Part II Version 1.1 amendments to complete the three-part enforcement authority.

Normative incorporation rule: the Version 1.0 body remains in force in full except where this Version 1.1 amendment explicitly adds, clarifies, or supersedes a clause. Part III operationalizes Parts I and II; it does not relax their proof requirements or authorize product behavior changes.

1. Amendment Summary and Authority Order

Part I defines whether prior proof remains semantically valid. Part II governs how scarce validation resources are queued, isolated, and handed off. Part III requires repository automation, Codex tasks, acceptance records, and release decisions to apply those rules consistently.

Authority	Binding effect
Part I v1.1	Semantic invalidation, immutable carry-forward, test-only ceilings, failure-class repair scope, and environment-qualified evidence.
Part II v1.1	Broker-owned fair queue, pre-acquisition source guard, automatic handoff, isolated lanes, and pressure routing.
Part III v1.1	Task, acceptance, closure, and release enforcement of Parts I and II; proportionate validation and incident detection.

2. Mandatory Task Decision Sequence

- Classify the change as material product, test-only, evidence-only, policy, or environment/infrastructure before selecting repair or validation scope.
- Create or update an impact and Evidence Invalidation Manifest before discarding prior receipts; SHA difference alone is not invalidation.
- Carry forward only immutable prior receipts whose producer, consumer, contract, fixture, policy, resource, and environment dependencies remain unchanged; record the new source-bound carry-forward receipt.
- Apply the Part I test-only invalidation ceiling. A broader rerun needs a written dependency explanation; cost, ritual, or unease is not a sufficient reason.
- Record the failure classification before source mutation whenever it determines the legal repair owner or scope.
- Request scarce resources through the Part II broker. Do not coordinate queue order, seize a lease, delete a lock, or bypass a waiting request through chat or task text.
- Run the smallest authoritative evidence that resolves every invalidated mandatory obligation. Preserve valid evidence and name any unavailable external proof accurately.

3. Failure Classification Controls Repair Scope

Classification	Required repair boundary	Prohibited shortcut
PRODUCT_DEFECT	Smallest responsible production scope, regression coverage, and affected evidence.	Changing only the test to hide a product failure.
TEST_DEFECT / STALE_TEST	Owning assertion, locator, fixture, or contract migration plus direct evidence consumers.	Changing product behavior without independent product-defect evidence.
ENVIRONMENT_DEFECT / TIMEOUT	Qualified environment, provisioning, pressure routing, or diagnosed budget correction.	Treating local contention or a queue wait as product failure.
INFRASTRUCTURE_DEFECT / CONTENTION	Broker, executor, lease, or infrastructure-owner boundary.	Feature refactor, lock deletion, or queue bypass.
EXTERNAL UNAVAILABLE	Record unavailable status and schedule/reference the required external proof.	Converting absence of a provider or hosted runner into a pass.

4. Evidence Preservation and Revalidation

A task may not rerun an unrelated full gate merely because the branch advanced, an index regenerated, a receipt was rewoded, or a test-only repair changed the source SHA. The planner must show a

dependency change for every invalidated receipt. A valid acceptance record is convergent: once all invalidated obligations have fresh or preserved producers, further reruns require a newly detected semantic dependency, changed gate requirement, failed evidence-integrity check, or closure-revalidation incident.

Disposition	Task/release treatment
PRESERVED	Use only through a new source-bound carry-forward receipt and dependency fingerprint.
REBOUND	Regenerate or rebind source-bound record identity; preserve semantic proof.
INVALIDATED	Run or replace the affected evidence before acceptance.
SUPERSEDED	Retain historical receipt; use the stronger valid replacement for decision purposes.

5. Prompt Convergence and Full-Gate Prohibition

After valid acceptance evidence exists, Codex and automation must converge to closure rather than keep searching for a larger ritual. Do not schedule an enormous product, browser, or release matrix for a documentation-only, evidence-only, or test-only change unless the impact analysis identifies a changed dependency that makes that gate mandatory. Track repeated full gates, over-invalidation, queue wait, and mainline-drift time as verification-quality signals.

- A request for confidence is not, by itself, authority to discard valid evidence.
- A broad gate may be selected when a product, schema, security/privacy, release-policy, shared runtime, or cross-project contract dependency changed and the planner names that path.
- When a source guard detects stale input, cancel before expensive acquisition, reconcile outside the lane, and create a new request; do not run stale proof merely to satisfy a queue position.
- Queue delay, cold-start delay, and execution time are different facts and must not be reported as one product timeout.

6. Closure-Revalidation Incident Detection

Closure revalidation is a targeted integrity check, not a second release ceremony. It is required when a post-acceptance signal indicates that the accepted decision may no longer resolve the same obligations.

Incident signal	Required response
Changed protected contract, schema, policy, fixture, environment identity, or resource contract	Open an Evidence Invalidation Manifest and revalidate only affected obligations.

Incident signal	Required response
Receipt digest mismatch, missing artifact, contradictory finalizer input, or unverified carry-forward	Treat the evidence as invalid until integrity is restored; investigate the receipt chain.
Late root failure previously marked cascade/blocked	Reclassify, identify the legal repair scope, and reopen only the dependent decision boundary.
Hosted/reference reproduction contradicts local timing or provider evidence	Qualify or invalidate the affected timing/provider claim; do not erase functional evidence without dependency analysis.
No semantic incident	Do not reopen closure or rerun a full gate.

7. Shared Validation Queue and Reference Environments

All scarce browser, build, provider, database, scanner, and restart-host work uses the shared broker queue. Fairness age prevents starvation; active work is not ordinarily preempted; compatible fully isolated lanes may run concurrently; cleanup completes before automatic handoff. A candidate that becomes stale while waiting is cancelled before expensive resource acquisition.

Hosted clean runners are reference environments for timing-sensitive, browser, capacity, and provider claims. A contended developer workstation may produce functional diagnostics, but its timing result is qualified. External/provider absence remains EXTERNAL BLOCKED or UNAVAILABLE, never passed by substitution.

8. Record-Only Closure and Proportionate Validation

For documentation, index, receipt, evidence-record, or catalog-status changes that do not alter executable product or release-gate semantics, closure may be record-only. Record-only closure requires the smallest authoritative validation for the changed records: document/index/policy validation, link and schema checks, generator output where owned by a generator, and a proportionate finalizer or policy check if that record feeds a decision. It does not authorize product behavior, browser, or release-matrix reruns absent a proven dependency.

- Regenerate, do not hand-edit, generator-owned indexes and catalogs.
- Preserve the Feature Catalog unless a completed major product capability, meaningful availability change, or limitation changed under its own governance.
- Name validation as passed, skipped-by-policy, unavailable, blocked, or inherited-broken; never represent a non-run as a pass.
- The final closure record must identify preserved, rebound, fresh, invalidated, unavailable, and excluded evidence separately.

9. Part III v1.1 Acceptance Additions

- Tasks classify failure before selecting repair scope and cannot use a test edit to conceal a product defect.
- Semantic evidence invalidation and carry-forward are auditable task obligations.
- Test-only invalidation ceilings prevent unrelated full-gate reruns without a dependency explanation.
- Valid acceptance evidence causes prompt convergence unless a defined closure-revalidation incident occurs.
- Shared validation fairness, automatic handoff, and source guards replace conversational lane ownership.
- Hosted/reference environments qualify timing and provider claims; local contention is not feature evidence.
- Record-only governance changes close with proportionate policy/document validation rather than release-matrix ritual.

R. References and Amendment Closure

Base authority: Project Sounding Line Part III - Repository, Codex, and Release Enforcement, Version 1.0, July 28, 2026. Companion authority: Parts I and II Version 1.1 Amendment Editions. This Part III amendment is derived under owner authorization from the operative Part I and Part II amendments to complete the three-part v1.1 enforcement set.

All unaffected Version 1.0 clauses remain governing. Where Version 1.0 conflicts with this amendment on semantic invalidation, repair scope, queue governance, closure revalidation, reference environments, or proportionate record-only validation, Version 1.1 controls.